PathFinder

Every learning platform can tell you which courses are relevant. None of them tells you which to take *first*, which you aren't ready for, or which you can skip because you already know it. PathFinder answers that — and shows its work.

`Runs fully offline · no API key required` `FastAPI + React` `238 catalog items` `77 tests` `Reproducible build`

1

PROBLEM UNDERSTANDING

Relevance is solved. Sequence isn't.

A recommender answers *"what relates to this topic?"* A learner needs the answer to something harder: *"given what I already know, what should I do next, in what order, to reach a goal I can only describe vaguely?"*

That reframing is the whole design. It demands four things similarity search cannot provide on its own:

A model of what you already know

So the system stops recommending it — and so "relevant" can be measured against a starting point, not from zero.

A model of what the goal requires

A career target as a weighted skill vector turns a vague ambition into a measurable distance.

Ordering

A course you aren't ready for is worse than useless. Prerequisites are a graph, and the path must respect it.

Explanations

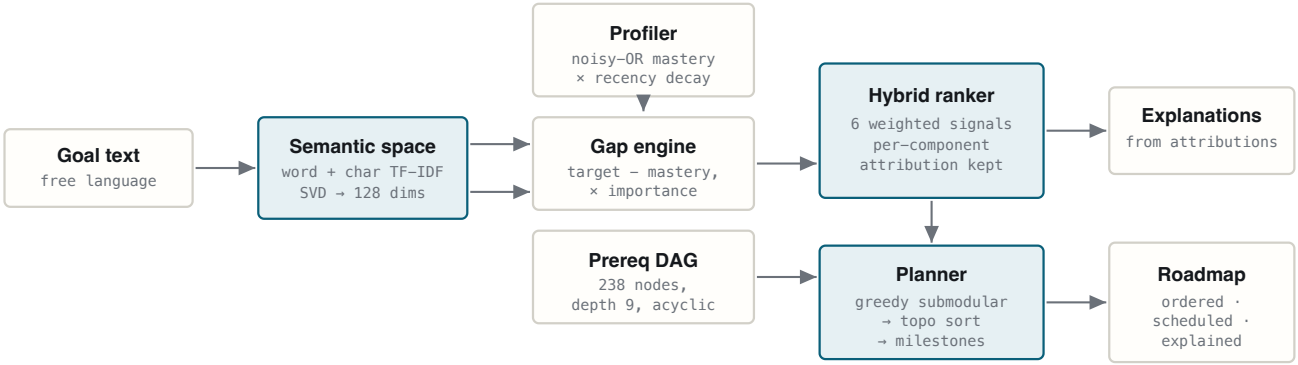
A learner who doesn't trust the ordering abandons it. The reasoning has to be inspectable, not asserted.

2

SOLUTION APPROACH

A pipeline, not a search box

A learner describes a goal in plain language. PathFinder resolves it to a career profile, estimates current mastery from their history, computes the weighted gap, ranks candidates on six independent signals, then plans an ordered, scheduled roadmap of courses, projects and assessments — attaching to each one the arithmetic that put it there.



The ranker's component attributions feed the explanation layer directly — the same numbers that produce the ordering produce the justification.

3

AI / ML TECHNIQUES

Six models, each covering another's blind spot

Representation — shared latent space

Items, skills, career roles and free-text goals are embedded into one 128-dimensional space so any two can be compared with a dot product. TF-IDF over hand-built documents, reduced by truncated SVD — latent semantic analysis. Two feature views are unioned before the reduction: **word 1–2 grams** carry topic, **character 3–5 grams** carry morphology.

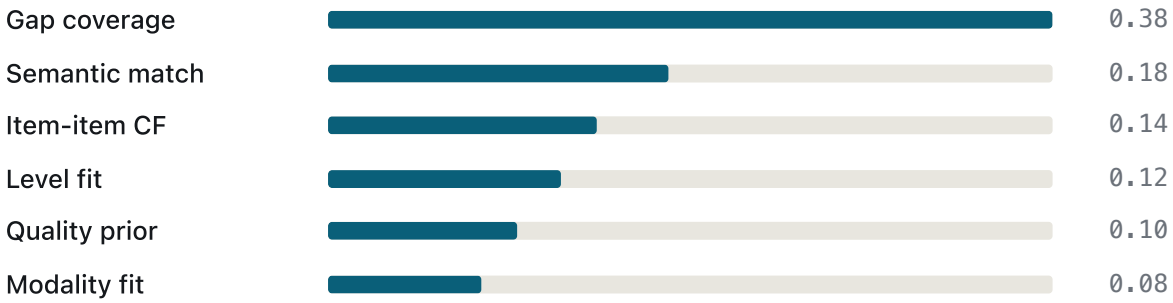
> WHY THE CHARACTER VIEW EXISTS
>
> With word features alone, the goal *"designing beautiful websites"* produced a **zero vector** — none of those word forms appear in the catalog — and the learner received no recommendations at all. Character n-grams reach "web design" through shared substrings. A regression test now guards it.

Learner profiling — noisy-OR mastery

Evidence combines multiplicatively rather than additively: `mastery = 1 − Π(1 − contributionᵢ)`. Summing would be wrong — two courses each teaching Python at 0.5 do not make an expert, they overlap. Noisy-OR produces genuine diminishing returns and cannot exceed 1.0, which is precisely what makes the gap engine stop asking for more Python once you clearly know Python.

DISCOUNT FACTOR	EFFECT
Depth	Advanced items teach more deeply — 0.85 / 1.0 / 1.15 by level
Recency	`0.5^(months/24)`, floored at 0.35 — skills fade, they don't vanish
Score	Assessment results are direct evidence; 0.85 assumed for a bare completion
Kind	Projects and assessments count 1.1× — building beats watching

Ranking — six weighted signals



Each signal fails alone: coverage ignores quality and readiness, semantic ignores what you know, collaborative filtering cold-starts and echoes popularity, level fit ignores the goal entirely. Two details do real work — the quality prior is **Bayesian-smoothed**, so a 4.9 from 200 raters can't outrank a 4.8 from 200,000; and CF similarity is **shrunk** by `co/(co+12)`, so a pair seen twice isn't trusted like a pair seen two hundred times. MMR re-ranking then trades a little relevance for breadth, because the top five would otherwise be five near-identical intro courses.

Path planning — budgeted maximum coverage

Choosing a path is a budgeted max-coverage problem, and the objective is **submodular** — each additional course on a topic adds less than the last. Greedy selection therefore carries the standard `(1 − 1/e)` approximation guarantee and runs in milliseconds. After every pick the planner **simulates the mastery the learner would gain** using the profiler's own noisy-

OR rule and recomputes the remaining gap — which is what stops it stacking four overlapping intro-ML courses. The selection is then closed under prerequisites, topologically sorted with Kahn's algorithm, and chunked into milestones on cumulative hours.

Explanation — derived, never narrated after the fact

The explanation layer reads the **same component attributions that produced the ranking**. Every sentence traces to a number the ranker used. This is an architectural constraint, not a stylistic one: a system that ranks with one model and explains with another can produce fluent, confident, wrong justifications. Here the explanation is computed *from* the ranking, so it cannot drift from it.

> ON THE OPTIONAL CLAUDE INTEGRATION
>
> With an API key, Claude improves intent extraction and rewrites explanations into warmer prose — given the computed facts and instructed to add none. Claude is **never** asked to choose or order recommendations; that stays deterministic and reproducible. Every call fails soft: no key, no package, no network, a rate limit or a malformed response all fall back silently to the local path. **The application is fully functional with no key at all.**

4

KEY FEATURES & WORKFLOWS

Four turns to a roadmap

The conversation is slot-filling rather than open-ended, because planning needs five specific things. The assistant asks only for what's still missing and stops the moment it has enough — and a single sentence can fill four slots at once.

TURN	LEARNER SAYS	WHAT THE SYSTEM DOES
1	"I want to become a generative AI engineer"	Resolves to a role profile — 14 weighted target skills. Ambiguous goals produce a clarifying question, not a guess.
2	"some grounding"	Sets the difficulty anchor for level-fit scoring
3	"I did Python for Everybody and Prompt Engineering for Developers"	Token-overlap matches both against the real catalog; they become mastery evidence, not text
4	"15 hours a week, hands-on"	Sets the schedule and the modality preference; the path generates

289 h	19.3 w	84%	10→89%
PLANNED EFFORT	AT 15 H / WEEK	GAP COVERED	GOAL READINESS

Actual output for the four turns above — five milestones, prerequisites pulled in automatically.

The projects guarantee

Coverage-per-hour has a blind spot: a 20-hour project consolidating four already-taught skills scores worse than two 10-hour courses each introducing a new one. Left alone, the planner produced **a reading list with no hands-on work at all**. Projects and assessments now carry a selection bonus and a minimum of two projects is guaranteed. A path you can't show anyone isn't a path.

Adaptation

Feedback folds back into the *profile* and the path regenerates — it is never patched in place. One source of truth means the roadmap cannot drift from the learner model, and regeneration costs milliseconds.

SIGNAL	EFFECT ON THE PLAN
Completed (with score)	Credited as mastery; a low score credits only partially, so reinforcement can reappear
Too easy	Credits the skills and raises the difficulty target
Too hard	Lowers the target and inserts groundwork — the item is not dropped
Not for me	De-emphasises the topic, but keeps what the goal still genuinely requires
Pace change	Rescales the entire schedule and every milestone date

5

DATA

Curated, committed, verified at build time

238	116	22	9
CATALOG ITEMS	SKILLS · 13 CATEGORIES	CAREER PROFILES	MAX PREREQ DEPTH

190 courses, 27 projects, 21 assessments from real providers — Coursera, edX, freeCodeCamp, NPTEL, DeepLearning.AI, Hugging Face, AWS and others. Curated rather than scraped for three reasons: **prerequisite edges exist in no public course dataset** and are the backbone of the system; a committed catalog means the demo cannot fail on a network call; and correctness becomes enforceable.

The build refuses to emit output if any prerequisite dangles, any skill is unknown, any id repeats, or the graph contains a cycle. Every skill is taught by at least two items, so the ranker always has a real choice. Rebuilding from source produces **byte-identical** artifacts — verified.

> STATED PLAINLY
>
> We have no real enrolment data, so the collaborative-filtering component is fitted on **4,000 simulated sessions** generated from a fixed seed. It is a genuine item-item CF model over genuine co-occurrence structure — but the co-occurrences are synthetic. It carries 14% of the ranking weight; the other 86% uses no simulated data.

6

CHALLENGES FACED

Bugs that changed the design

Each was found by using the thing — driving the conversation the way a student actually would, and screenshotting the running app. All are now covered by regression tests.

A learner's goal was silently recorded as their history

SYMPTOM — "I want to become a machine learning engineer" matched the course title *Intro to Machine Learning* by token overlap and was logged as completed — so the system removed from the path the very thing the learner asked to learn.

ROOT CAUSE — Title matching had no notion of whether the sentence was a claim about the past or a statement of intent.

FIX — Matching is gated on a completion cue ("did", "finished", "already know"). The gate lifts only when the assistant has just asked what they've completed, since the reply is then a bare list of titles.

Foundations were scheduled after the material built on them

SYMPTOM — Linear Algebra appeared *after* the ML Specialization that requires it — technically valid under the DAG, pedagogically backwards.

ROOT CAUSE — Prerequisites pulled in as fillers carried zero ranking priority, so they sorted to the back of the path and dragged their dependents with them.

FIX — Priority propagates backwards through the graph: a prerequisite is exactly as urgent as the most urgent thing it unlocks.

Answers to questions were mined as learning goals

SYMPTOM — A learner asking to learn AI received a *JavaScript* path.

ROOT CAUSE — Every message was scanned for goal intent, including replies to our own questions. "nothing" returned algorithms, GraphQL and cryptography from the embedding's weak tail; "6 hours a week" returned React. Those accumulated as goals and outvoted the real one.

FIX — Replies to a prompt are never mined for intent, and skill-based goals require an explicit match rather than the tail.

Nonsense produced a confident career path

SYMPTOM — "asdkjh qwe zxcv" generated a 423-hour Computer Vision Engineer roadmap.

ROOT CAUSE — The character n-grams that make unseen word forms work also match noise: "z**xcv**" reaches "cv" reaches computer vision, scoring above the commit threshold.

FIX — A separate word-level check asks whether the learner used any vocabulary the catalog recognises. With none, the ranking is character noise, so the assistant offers real choices instead of inventing a career.

Unseen word forms returned nothing at all

SYMPTOM — "designing beautiful websites" produced an empty recommendation list — not a bad one, an empty one.

ROOT CAUSE — Word-level TF-IDF has no entry for "websites" or "designing", so the query vectorised to all zeros.

FIX — Character n-grams unioned into the feature space, plus an explicit empty-vector check so "no signal" is handled as a case rather than scored as zero similarity to everything.

7

ENGINEERING

Verification & limits

77	~2 s	1
TESTS PASSING	FULL SUITE	PROCESS TO DEPLOY

The route layer contains no reasoning — every engine is a pure function over a profile and the catalog. That's why the algorithms can be tested directly without HTTP, and why the same profile always yields the same path. Tests cover catalog invariants (acyclicity, skill coverage, no dangling edges), mastery saturation and decay, topological correctness, ranking determinism, path construction, and the complete HTTP flow.

Verified from a clean environment on **both Python 3.10 and 3.12**: unzip → install → build catalog → 77 tests pass → frontend builds → artifacts byte-identical to the committed ones. That last check earned its place — it caught CPython 3.12 switching `sum()` over floats to compensated summation, which shifted role-overlap scores in their final bits and flipped near-ties in a sort. The build now uses `math.fsum` with explicit rounding and a deterministic tie-break, and a test fails if reproducibility ever regresses.

What we'd fix with more time

- **Ranking weights are hand-set, not learned.** With no real engagement data there's nothing to fit them against; they were tuned against the behaviours in the test suite.
- **LSA under-performs a neural sentence encoder** on paraphrase-heavy input. The character view narrows that gap; it doesn't close it. We chose reproducibility and a zero-download install over the last few points of accuracy.
- **238 items is a prototype catalog.** The ranker is vectorised and the graph algorithms are near-linear, but retrieval would need an ANN index beyond ~10^5 items.

> - **Sessions are in-memory**, capped at 500 and cleared on restart. Swapping one module for Redis is the only change required.
> - **Mastery is inferred, not measured.** Finishing a course is weak evidence of having learned it — which is exactly why assessments are first-class catalog items rather than an afterthought.

PathFinder · AI-Powered Personalized Learning Path Recommender

Team Critical Path — R Koushik · Chethan H S · Khushi Katwe · Sandeep N · Chethan Kumar H M

REVA University · HCLTech Round 2